

Day 11: Store Transaction in a List

1. Day Number

Day 11

2. Topic Name

Lists and append()

Today you will learn how to:

- Create a list
 - Add an item using `append()`
 - Store transaction information as text
 - Print the complete list
-

3. Connection with Yesterday

Previously, you learned how to deposit money and update the account balance.

Today, you will take the next step:

Instead of only updating the balance, you will also save the deposit information in a transaction history.

For example:

```
Balance: $500  
  
Transaction history:  
["Deposit $100"]
```

Later, this list can store multiple transactions:

```
["Deposit $100", "Deposit $50", "Withdrawal $20"]
```

4. Important Topics

List

A **list** stores multiple values inside one variable.

Lists use square brackets:

```
transactions = []
```

This creates an **empty list**.

A list with values looks like this:

```
transactions = ["Deposit $100", "Deposit $50"]
```

append()

The `append()` method adds one new item to the end of a list.

```
transactions.append("Deposit $100")
```

After this statement, the list becomes:

```
["Deposit $100"]
```

String Formatting

String formatting allows you to combine text and a variable.

An f-string is written by placing `f` before the quotation marks:

```
amount = 100
message = f"Deposit ${amount}"
```

The value of `message` will be:

```
Deposit $100
```

5. Foundational Notes

A list can start empty

```
transactions = []
```

This is useful when you do not have any transactions initially.

A list can store strings

```
transactions = ["Deposit $100"]
```

Here, "Deposit \$100" is a string stored inside the list.

append() changes the original list

```
transactions.append("Deposit $50")
```

You do not need to create another list. Python adds the item directly to the existing list.

User input is initially a string

```
amount = input("Enter deposit amount: ")
```

Even when the user types 100, Python initially receives it as text.

For calculation, convert it into a number:

```
amount = float(amount)
```

For this exercise, either `int()` or `float()` may be used depending on whether decimal deposits are allowed.

A formatted transaction is still a string

```
transaction = f"Deposit ${amount}"
```

The amount may be a number, but the complete transaction message is text.

6. Easy Example

Suppose you want to store a completed task:

```
tasks = []

task_name = "Complete Python practice"
tasks.append(task_name)

print(tasks)
```

Output:

```
['Complete Python practice']
```

What happened?

1. An empty list was created.
2. A task was stored in a variable.
3. `append()` added the task to the list.
4. The updated list was printed.

Your transaction problem follows the same idea.

7. Problem Statement

Create an empty transaction list.

Ask the user to enter one deposit amount.

Create transaction text in this format:

```
Deposit $100
```

Add that transaction text to the list using `append()`.

Finally, print the transaction list.

8. Concepts Used

You will use:

- Variables
- Empty lists

- `input()`
 - Number conversion using `int()` or `float()`
 - f-strings
 - `append()`
 - `print()`
 - Functions
-

9. Thought Process

Think about the problem in small steps.

Step 1: Create storage

You need somewhere to store transaction history.

Therefore, create an empty list:

```
transactions = []
```

Step 2: Get the deposit amount

Ask the user to enter the amount.

Example:

```
Enter deposit amount: 100
```

Step 3: Convert the input

The input comes as a string, so convert it into a number.

```
"100" → 100
```

Step 4: Create the transaction message

Combine "Deposit \$" and the entered amount.

```
Deposit $100
```

Step 5: Add the message to the list

Use `append()` to store the transaction.

Before:

```
[]
```

After:

```
["Deposit $100"]
```

Step 6: Print the list

Display the complete transaction history.

10. Pseudocode

```
START  
  
DEFINE a function to store a deposit transaction  
  
    CREATE an empty transaction list  
  
    ASK the user for the deposit amount  
  
    CONVERT the input into a number  
  
    CREATE a formatted transaction message  
        "Deposit $amount"  
  
    ADD the transaction message to the list  
  
    PRINT the transaction list  
  
CALL the function  
  
END
```

11. Suggested Solving Approach: Functional Approach

Put the main logic inside a function.

Suggested structure:

```
def store_deposit_transaction():  
    # Create an empty list  
    # Ask for the deposit amount  
    # Create the transaction message  
    # Append the message  
    # Print the list
```

Then call the function:

```
store_deposit_transaction()
```

Why use a function?

A function:

- Keeps the program organised
- Groups related instructions together
- Makes the code reusable
- Prepares you for larger banking programs

12. Easy Edge Cases

Empty List

At the beginning, the transaction list should be empty:

```
[]
```

After adding one deposit, it should contain one item:

```
["Deposit $100"]
```

Deposit 0

If the user enters:

```
0
```

The program may store:

```
["Deposit $0"]
```

For today's simple list practice, storing zero is acceptable.

Later, you can add validation so that only positive amounts are stored.

Decimal Deposit

If the user enters:

```
25.50
```

Using `float()` allows this value.

Possible transaction:

```
["Deposit $25.5"]
```

Later, you can learn formatting to always show two decimal places:

```
Deposit $25.50
```

13. Expected Input

Example:

```
Enter deposit amount: 100
```

The user enters one deposit amount.

14. Expected Output

Example:

```
['Deposit $100']
```

Another possible example:

```
Enter deposit amount: 50  
['Deposit $50']
```

The exact number format may differ depending on whether you use `int()` or `float()`.

For example, using `float()` might produce:

```
['Deposit $50.0']
```

15. Hint Only

Start with an empty list:

```
transactions = []
```

Create the transaction message using an f-string:

```
transaction = f"Deposit ${...}"
```

Then use:

```
transactions.append(...)
```

Finally, print the list.

Your main missing pieces are:

- What should replace `...` inside the f-string?
- What value should be passed to `append()`?